

EMILI-IMRT

Documentación Técnica Completa

Optimización de Radioterapia de Intensidad Modulada

Framework EMILI + Subproblemas BAO y FMO

Beam Angle Optimization · Fluence Map Optimization

Metaheurísticas · OSQP · Estructuras de Vecindad · Gramáticas de Configuración

Resumen del Sistema

Plataforma:	C++ 11/14, CMake, OSQP
Paradigma:	Metaheurísticas + QP Exacto
Dominio:	Planificación de Tratamiento de RT
Configuración:	Gramáticas XML + irace/GGA

Tabla de Contenidos

1	Introducción y Contexto Clínico	3
1.1	Radioterapia de Intensidad Modulada	3
1.2	Problemas de Optimización en IMRT	3
2	Arquitectura General del Sistema	4
2.1	Estructura de Archivos	4
2.2	Pipeline de Ejecución	4
3	Framework EMILI	5
3.1	Clases Base del Framework	5
3.2	Algoritmos Metaheurísticos	6
3.3	Criterios de Terminación	7
4	Representación de la Instancia IMRT	8
4.1	Carga de Datos	8
4.2	Estructuras de Datos Clave	8
4.3	Formatos Soportados	9
5	Subproblema FMO (Fluence Map Optimization)	10
5.1	Formulación del Problema	10
5.2	Solución Inicial	10
5.3	Estructuras de Vecindad FMO	11
5.4	Perturbaciones y Aceptación	11
6	Subproblema BAO (Beam Angle Optimization)	12
6.1	Formulación del Problema	12
6.2	Solución Inicial BAO	12
6.3	Vecindad de Intercambio de Ángulos	13
6.4	Solver OSQP para FMO Exacto	13
7	Constructor de Componentes (ImrtBuilder)	15
7.1	Palabras Clave Disponibles	15
8	Gramáticas de Configuración (irace/GGA)	16
8.1	Estructura de Gramática	16
8.2	Parámetros Numéricos Configurables	17
9	Flujo de Ejecución Detallado	18
9.1	Flujo BAO+FMO Completo	18
9.2	Reportes Clínicos ICRU-83	19
10	Resumen de Diseño y Patrones	20

1 Introducción y Contexto Clínico

1.1 Radioterapia de Intensidad Modulada (IMRT)

La Radioterapia de Intensidad Modulada (IMRT) es una técnica avanzada de tratamiento oncológico que permite administrar dosis de radiación con alta precisión espacial. El objetivo es maximizar la dosis absorbida en el volumen tumoral (PTV, Planning Target Volume) mientras se minimizan los daños en los órganos de riesgo adyacentes (OARs, Organs At Risk). Para ello, un acelerador lineal gira alrededor del paciente a distintos ángulos de gantry, emitiendo haces subdivididos en pequeñas bixeles denominados beamlets. Las intensidades de estos beamlets son las variables de decisión del problema de optimización.

1.2 Problemas de Optimización en IMRT

El proceso de planificación de tratamiento involucra dos subproblemas estrechamente relacionados:

- FMO - Fluence Map Optimization: dado un conjunto fijo de K ángulos de gantry, optimizar las intensidades continuas de los beamlets para minimizar las penalizaciones de subdosis en PTV y sobredosis en OARs.
- BAO - Beam Angle Optimization: seleccionar el conjunto óptimo de K ángulos de gantry de entre n candidatos, de manera que la FMO resultante sea lo más eficaz posible. Es un problema combinatorio NP-difícil ($C(n,K)$ subconjuntos posibles).

Jerarquía de Optimización

BAO (combinatorio, exterior) -> selecciona K ángulos

FMO (continuo cuadrático, interior) -> optimiza intensidades para esos K ángulos

La combinación de ambos subproblemas se resuelve mediante metaheurísticas (BAO) con solución exacta del FMO vía OSQP en cada evaluación.

2 Arquitectura General del Sistema

2.1 Estructura de Archivos del Proyecto

Archivo / Directorio	Descripción
main.cpp	Punto de entrada; parseo de argumentos y ejecución
emilibase.h/cpp	Framework EMILI: clases base para metaheurísticas
generalParser.h/cpp	Parser de configuración y tokens de línea de comandos
imrt/imrt.h/cpp	Problema FMO, soluciones, vecindades de beamlets
imrt/imrt_instance.h/cpp	Carga de instancias, cálculo de dosis, reportes ICRU-83
imrt/imrt_builder.h/cpp	Fábrica de componentes IMRT desde tokens CLI
imrt/imrt_bao.h/cpp	Problema BAO: selección de ángulos, vecindad de intercambio
imrt/imrt_fmo.h/cpp	Solver OSQP: resolución exacta del QP de FMO
imrt_grammar*.xml	Gramáticas XML para configuración automática con irace/GGA
CMakeLists.txt	Configuración de compilación (WITH_OSQP, flags, targets)

2.2 Pipeline de Ejecución

El flujo de ejecución desde la línea de comandos es el siguiente:

Invocación desde CLI

```
./emili_imrt [problema] [solución_inicial] [metaheurística] [parámetros]
```

Ejemplos:

BAO con ILS + intercambio de ángulos:

```
./emili_imrt baoimrt 5 instances/PROSTATE/ ifirstk nangswap ils locmin prangswap aimprove
```

FMO con búsqueda por primera mejora + ILS:

```
./emili_imrt imrt instances/PROSTATE/ izero nshift 0.5 ils locmin prandom 5 10 aimprove
```

El componente GeneralParserE tokeniza los argumentos y los redirige a EmBaseBuilder (componentes del framework) o ImrtBuilder (componentes específicos de IMRT). Una vez contruidos todos los componentes, se ejecuta el algoritmo con `ls->timedSearch(seconds)` o `ls->search()`, y al final se genera el reporte clínico.

3 Framework EMILI (emilibase.h/cpp)

3.1 Clases Base Abstractas

EMILI define una jerarquía de clases abstractas que representan los bloques funcionales de cualquier metaheurística. Cada componente es una estrategia intercambiable:

Clase Abstracta	Responsabilidad	Método Principal
Problem	Evalúador de función objetivo	calcObjectiveFunctionValue(Solution&)
Solution	Contenedor de una solución	clone(), isFeasible(), getSolutionRepresentation()
InitialSolution	Generador de solución inicial	generateSolution(), generateEmptySolution()
Neighborhood	Generador de movimientos/vecinos	computeStep(Solution*), reverseLastMove(Solution*)
Perturbation	Mecanismo de escape de mínimo local	perturb(Solution*)
Acceptance	Criterio de aceptación de solución	accept(current, candidate)
Termination	Condición de parada	isSatisfied()
LocalSearch	Motor de búsqueda local	search(), timedSearch(seconds)

3.2 Algoritmos Metaheurísticos Implementados

>> Búsqueda Local Simple

- FirstImprovementSearch: acepta el primer vecino que mejore la solución actual.
- BestImprovementSearch: explora toda la vecindad y acepta el mejor vecino.
- TieBrakingBestImprovementSearch: desempate con un objetivo secundario.

>> Iterated Local Search (ILS)

Estructura: LocalSearch + Perturbation + Acceptance. Flujo:

Pseudocódigo ILS

```
s = initial_solution()
s = local_search(s)
loop until termination:
    s' = perturb(s)
    s' = local_search(s')
    s = accept(s, s')
return best_so_far
```

>> Tabu Search

- BestTabuSearch / FirstTabuSearch: búsqueda local con memoria tabú.
- ImrtTabuMemory: buffer circular de vectores de intensidad con tenure configurable.
- GeneralTabuSearch: envuelve cualquier LocalSearch con restricciones tabú.

>> Variable Neighborhood Descent / Search (VND/VNS)

- VNDSearch: alterna entre vecindades; si mejora en vecindad k -> resetear a k=0, si no -> k++.
- GVNS (General VNS): combina shake (perturbación) + NeighborhoodChange (aceptación con incremento de k).
- PerShake: usa objetos Perturbation como shakes para VNS.

>> Simulated Annealing

- MetropolisAcceptance: temperatura fija, criterio de aceptación estocástico.
- Metropolis: esquema de enfriamiento progresivo con parámetros start_temp, end_temp, temp_ratio.

3.3 Criterios de Terminación

Clase	Condición de Parada
-------	---------------------

LocalMinimaTermination	Detiene al no encontrar mejora (mínimo local)
WhileTrueTermination	Nunca termina (uso con timer externo)
TimedTermination	Detiene al superar tiempo de pared dado
MaxStepsTermination	Detiene tras N iteraciones
MaxEvaluations	Detiene tras N evaluaciones de función objetivo
MaxStepsOrLocmin	Combinado: N pasos O mínimo local
ImrtMaxIterations	Cuenta intercambios de ángulos (específico para BAO)
ImrtFeasibleTermination	Detiene si objetivo $\leq$ tolerancia (convergencia a factibilidad)

4 Representación de la Instancia IMRT (imrt_instance.h/cpp)

4.1 Carga de Datos

La clase `ImrtInstance` gestiona la carga y organización de los datos del problema. Soporta dos formatos de archivo distintos:

- Formato clásico: un archivo de texto por cada par (órgano, ángulo) con nombre `ORGANNAME_<ángulo>.txt`, más un `instance_config.txt` que lista ángulos, órganos, pesos y restricciones.
- Formato CORT: archivos `VOIList` por órgano y matrices `Dij` completas por ángulo (`Gantry<g>_Couch<c>_D.txt`), con detección automática del formato.

El archivo de configuración `instance_config.txt` acepta las siguientes directivas:

`instance_config.txt`

```
angles    0 40 80 120 160 200 240 280 320      # ángulos de gantry (grados)
dimlets   625                                # beamlets por ángulo
ptv       PROSTATE  Dmin=74.0                 # PTV con dosis mínima
oar       BLADDER   Dmax=72.0                 # OAR con dosis máxima
oar       RECTUM    Dmax=70.0 vol_frac=0.25 dose_limit=65.0
w_under   1.0                                           # peso subdosis PTV
w_over    0.1                                           # peso sobredosis OAR
max_intensity 10.0                                     # cota superior beamlets
```

4.2 Estructuras de Datos Clave

Estructura	Campos Principales	Descripción
<code>DoseEntry</code>	<code>boxet_id</code> , <code>dimlet_id</code> , <code>dose_rate</code>	Entrada de matriz dispersa: beamlet -> dosis en vóxel
<code>OrganData</code>	<code>name</code> , <code>is_ptv</code> , <code>Dmin</code> , <code>Dmax</code> , <code>entries[]</code>	Órgano con restricciones de dosis y matriz dispersa
<code>GridInfo</code>	<code>nx</code> , <code>ny</code> , <code>nz</code> , <code>dx</code> , <code>dy</code> , <code>dz</code> , <code>isocenter</code>	Geometría de la rejilla de vóxeles (solo formato CORT)
<code>ImrtInstance</code>	<code>n_angles</code> , <code>n_dimlets</code> , <code>organs[]</code> , <code>angles[]</code>	Clase contenedora principal del problema

4.3 Formatos de Instancia Soportados

Ambos formatos utilizan matrices de dosis dispersas (`DoseEntry`) para evitar el almacenamiento de la matriz `Dij` completa (que puede tener millones de entradas). La función `computeOrganDoses(x)` calcula el vector de dosis $d = D \cdot x$ recorriendo solo las entradas no nulas, con complejidad $O(\text{entradas_no_nulas})$.

5 Subproblema FMO - Fluence Map Optimization (imrt.h/cpp)

5.1 Formulación del Problema

La clase `ImrtProblem` implementa la función objetivo cuadrática penalizada para la optimización de mapas de fluencia:

Función Objetivo FMO (penalización cuadrática)

$$f(x) = w_{\text{under}} * \sum_{b \in \text{PTV}} \max(0, D_{\text{min}} - d_b)^2 + w_{\text{over}} * \sum_{o \in \text{OARs}} \sum_{b \in o} \max(0, d_b - D_{\text{max}})^2$$

donde $d_b = \sum_j D[b, j] \cdot x[j]$ (dosis en vóxel b)
 $x[j] \geq 0$ (intensidades no negativas)
 $x[j] \leq \text{max_intensity}$ (cota superior)

`ImrtSolution` almacena el vector de intensidades x in $\mathbb{R}^n$ ($n = n_{\text{angles}} \times \text{dimlets_per_angle}$). El campo `active_angle_idxs_` de `ImrtProblem` permite restringir la optimización a un subconjunto de ángulos, lo cual es usado por el flujo BAO.

5.2 Soluciones Iniciales para FMO

Clase	Token CLI	Descripción
<code>ZeroInitialSolution</code>	<code>izero</code>	Todas las intensidades $x[j] = 0$
<code>UniformInitialSolution</code>	<code>iuniform <v></code>	Todas las intensidades $x[j] = v$ (respeta ángulos activos)
<code>RandomInitialSolution</code>	<code>irandom <max></code>	Intensidades $x[j] \sim U[0, \text{max}]$ (aleatorio uniforme)

5.3 Estructuras de Vecindad para FMO

>> `SingleBeamletShift` (token: `nshift <delta>`)

Mueve un beamlet a la vez modificando su intensidad en $\pm \text{delta}$, con clamping a $[0, \text{max_intensity}]$. Tamaño de vecindad: $2 \times n_{\text{dimlets}}$ (o $2 \times |\text{ángulos_activos}| \times \text{dimlets_por_ángulo}$ en modo BAO). Implementa exploración sistemática recorriendo todos los dimlets en ambas direcciones.

>> `BeamletSwap` (token: `nswap`)

Intercambia las intensidades de dos beamlets distintos. Tamaño de vecindad: $n_{\text{dimlets}} \times (n_{\text{dimlets}} - 1) / 2$. Útil para reorganizar la distribución de fluencia sin cambiar su suma total.

Ambas vecindades exponen la interfaz de iterador:

Iteración sobre la vecindad

```
for (auto it = nbh.begin(sol); it != nbh.end(); ++it) {
    Solution* neighbor = *it;
    if (neighbor->solution_value < best_val) { ... }
    nbh.reverseLastMove(sol); // deshacer si no se acepta
}
```

5.4 Perturbaciones y Criterio de Aceptación

Componente	Clase	Descripción
Perturbación	<code>RandomBeamletPerturbation</code>	k beamlets aleatorios $\rightarrow U[0, \text{max_intensity}]$
Aceptación	<code>ImrtImproveAccept</code>	Acepta solo si la nueva solución mejora la actual
Aceptación SA	<code>MetropolisAcceptance</code>	Acepta con prob. $\exp(-\Delta f/T)$ (Simulated Annealing)

6 Subproblema BAO - Beam Angle Optimization (imrt_bao.h/cpp)

6.1 Formulación del Problema BAO

El problema BAO busca el conjunto A^* subset $\{0, \dots, n-1\}$ de K ángulos que minimiza el valor óptimo del FMO correspondiente. Se formula como:

Formulación BAO

$$\min_{\{A \text{ subset } \{0, \dots, n-1\}, |A|=K\}} f^*(A) = \min_{\{x \geq 0\}} \text{FMO}(x, A)$$

Espacio de búsqueda: $C(n, K)$ subconjuntos posibles.

Ejemplo: $n=36$ ángulos candidatos, $K=5 \rightarrow C(36, 5) = 376,992$ subconjuntos.

Evaluación de cada subconjunto: resolver QP exacto mediante OSQP.

La clase `BaoProblem` encapsula este evaluador. Cada llamada a `calcObjectiveFunctionValue(s)` invoca `ImrtFmoSolver::solve(active_angles)`, que resuelve el QP con OSQP y retorna el valor óptimo f^* junto con el vector de intensidades óptimo x^* .

6.2 Soluciones Iniciales BAO

Clase	Token CLI	Descripción
<code>FirstKAnglesInit</code>	<code>ifirstk</code>	Selecciona los primeros K ángulos: $\{0, 1, \dots, K-1\}$
<code>RandomKAnglesInit</code>	<code>irandomk</code>	Fisher-Yates sobre todos los ángulos, toma K primeros y ordena

`BaoSolution` almacena: (1) `active_angles_`: vector de K índices de ángulo (base 0); (2) `angle_degrees_`: valores en grados para visualización; (3) `intensities_`: vector de n_{dimlets} con las intensidades óptimas (las inactivas = 0).

6.3 Vecindad de Intercambio de Ángulos (AngleSwapNeighborhood)

Cada movimiento consiste en reemplazar un ángulo activo por un ángulo inactivo. El tamaño de la vecindad es $K \times (n - K)$. Al generar un vecino, se invoca automáticamente el solver OSQP para obtener $f^*(\text{vecino})$. La interface implementada es:

AngleSwapNeighborhood

```
computeStep(base_sol):
    activo[i] <- inactivo[j]           // intercambio de ángulo
    -> llamar OSQP::solve(new_angles) // resolver FMO exacto
    -> retornar BaoSolution con nuevo conjunto y f*

reverseLastMove(sol):
    restaurar active_angles_ del estado previo (intensities quedan desactualizadas)

random(sol):
    seleccionar activo[i] e inactivo[j] al azar -> evaluar con OSQP
```

Nota sobre complejidad

Cada evaluación de vecino requiere resolver un QP cuadrático con OSQP. La variable `WITH_OSQP` debe estar activada en CMake para compilar el módulo BAO. El tiempo de solución de OSQP escala con el número de beamlets activos ($K \times \text{dimlets_per_angle}$) y el número de vóxeles en todos los órganos.

6.4 Perturbaciones para BAO

Clase	Token CLI	Descripción
<code>RandomAnglesPerturbation</code>	<code>prangswap</code>	Intercambia p ángulos activos aleatoriamente con el pool inactivo (Fisher-Yates para

6 Solver OSQP para FMO Exacto (imrt_fmo.h/cpp)

Formulación QP con Variables de Holgura

ImrtFmoSolver reformula el problema FMO como un Programa Cuadrático (QP) estándar añadiendo variables de holgura para linealizar las funciones `max()`:

Formulación QP para OSQP

```
Variables:  z = [x_activo in R^(K·m);  u_ptv in R^|PTV|;  v_oar in R^|OAR|]

min   w_under * ||u||2 + w_over * ||v||2

s.t.   D_ptv * x_activo + u  >= Dmin    (G4: subdosis PTV)
        D_oar * x_activo - v  <= Dmax    (G5: sobredosis OAR)
        0 <= x_j <= max_intensity      (G1: cotas beamlets)
        u, v >= 0                      (G2, G3: holguras no negativas)
```

Construcción del QP y Configuración de OSQP

En cada llamada a `solve(active_angles)`, se construye el QP de tamaño reducido (solo K ángulos activos) en formato CSC (Compressed Sparse Column):

- Matriz P (Hessiana): diagonal, con 0 en bloque x, `w_under` en bloque u, `w_over` en bloque v.
- Matriz A (restricciones): combina G1 (identidad sparse), G4 (filas PTV) y G5 (filas OAR).
- Vectores q, l, u: vector lineal cero (P ya es cuadrático), cotas inferiores/superiores.

Parámetros de OSQP configurados para alta precisión:

Configuración de OSQP

```
settings.verbose      = 0          // sin salida por consola
settings.warm_starting = 0         // reinicio limpio en cada llamada
settings.eps_abs      = 1e-6       // tolerancia absoluta
settings.eps_rel      = 1e-6       // tolerancia relativa
settings.max_iter     = 10000      // iteraciones máximas
settings.polishing    = 1          // pulido de solución para máxima precisión
```

Extracción e Integración de la Solución

Tras la resolución, se extrae `x*` del vector solución de OSQP y se mapea de vuelta al vector completo de `n_dimlets` (los beamlets de ángulos inactivos quedan en 0). La función retorna `{x_full, f*}`, que son almacenados en `BaoSolution`.

7 Constructor de Componentes - ImrtBuilder (imrt_builder.h/cpp)

7.1 Palabras Clave Disponibles

ImrtBuilder implementa el patrón Builder para construir todos los componentes IMRT a partir de tokens de línea de comandos. Hereda de la clase Builder del framework.

>> Palabras Clave de Problema

Token	Clase Creada	Parámetros
imrt <dir> [nactive K] [verbose]	ImrtProblem	dir: directorio de instancia; nactive K: restringir a K primeros ángulos
baomrt <K> <dir> [verbose]	BaoProblem	K: número de ángulos a seleccionar; dir: directorio de instancia; requiere W

>> Soluciones Iniciales

Token	Modo	Clase
izero	FMO	ZeroInitialSolution
iuniform <v>	FMO	UniformInitialSolution(v)
irandom <max>	FMO	RandomInitialSolution(max)
ifirstk	BAO	FirstKAnglesInit
irandomk	BAO	RandomKAnglesInit

>> Vecindades, Perturbaciones y Terminación

Token	Tipo	Descripción
nshift <delta>	Vecindad FMO	SingleBeamletShift con paso delta
nswap	Vecindad FMO	BeamletSwap: intercambio de dos beamlets
nangswap	Vecindad BAO	AngleSwapNeighborhood: intercambio de ángulos
prandom <k> <max>	Perturbación FMO	k beamlets aleatorios a U[0, max]
prangswap	Perturbación BAO	RandomAnglesPerturbation: swap de ángulos
aimprove	Aceptación	ImrtImproveAccept: solo mejoras
tmaxiter <n>	Terminación BAO	Detener tras n intercambios de ángulos
tfeasible	Terminación FMO	Detener si objetivo <= tolerancia

8 Gramáticas de Configuración XML (imrt_grammar*.xml)

8.1 Propósito y Estructura

Las gramáticas XML definen el espacio de configuración de algoritmos para herramientas de autoajuste como irace (R) o GGA (Genetic-Guided Algorithm). Cada gramática describe producciones BNF donde los valores numéricos tienen rangos y pasos configurables.

El sistema incluye cuatro archivos de gramática principales:

Archivo	Variante
imrt_grammar.xml	Gramática principal: todos los metaheurísticos
imrt_grammar_repair.xml	Terminación fija (maxstep 999-1000)
imrt_grammar_time.xml	Terminación basada en tiempo de pared
gramaticas/imrt_grammar_all.xml	Variante completa con todos los tipos

8.2 Parámetros Numéricos Configurables

Parámetro	Rango	Paso	Uso
delta	[0.1, 5.0]	0.1	Tamaño de paso en SingleBeamletShift
intensity	[0.5, 10.0]	0.5	Intensidad inicial uniforme
max_intensity	[1.0, 10.0]	1.0	Cota superior de beamlets
metrotemp	[0.1, 300]	1	Temperatura Metropolis (fija)
tabutenure_size	[1, 200]	1	Tamaño de la memoria tabú
random_moves	[1, 100]	1	Movimientos en perturbación aleatoria
start_temp	[1.1, 50000]	0.0001	Temperatura inicial SA
end_temp	[0, 1.0]	0.0001	Temperatura final SA
temp_ratio	[0.0001, 1]	0.0001	Ratio de enfriamiento SA
tempiterations	[1, 5000]	1	Iteraciones por temperatura SA
steps	[1000, 100000]	1000	Número máximo de pasos

8.3 Metaheurísticos Derivados

Token	Clase EMILI	Descripción
first	FirstImprovementSearch	Primera mejora
best	BestImprovementSearch	Mejor mejora
ils	IteratedLocalSearch	first/best + terminación + perturbación + aceptación
tabu	BestTabuSearch	Búsqueda tabú con memoria configurable
vnd	VNDSearch	VND con 3 vecindades (nshift 1, nshift 0.5, nswap)
vns	GVNS	VNS general con PerShake y NeighborhoodChange

9 Flujo de Ejecución Detallado

9.1 Flujo BAO + FMO Completo

Pseudocódigo BAO + FMO con ILS

INICIALIZACIÓN

```
1. ImrtInstance::loadFromDirectory(dir)
   |-- detectar formato (clásico / CORT)
   |-- cargar config: ángulos, dimlets, órganos, pesos
   |-- construir matrices de dosis dispersas (DoseEntry[])

2. ImrtFmoSolver(inst)
   |-- pre-computar índices PTV/OAR
   |-- pre-computar cotas fijas del QP

3. BaoProblem(inst, fmo, K)
```

SOLUCIÓN INICIAL

```
4. FirstKAnglesInit / RandomKAnglesInit -> BaoSolution(active=[0..K-1])
   |-- BaoProblem::evaluateSolution(s) -> OSQP::solve(angles) -> f*, x*
```

BÚSQUEDA (ejemplo ILS)

```
5. LocalSearch::search(initial_sol)
   loop until termination:
     a. AngleSwapNeighborhood::computeStep(s):
        |-- reemplazar activo[i] por inactivo[j]
        |-- OSQP::solve(new_angles) -> evaluar f*, x*
     b. Si mejora -> actualizar solución actual
     c. Si mínimo local:
        |-- RandomAnglesPerturbation::perturb(s) -> nueva configuración
        |-- LocalSearch::search(perturbed) -> nueva búsqueda local
     d. ImrtImproveAccept::accept(current, candidate)
```

RESULTADO

```
6. Reportar mejor BaoSolution:
   |-- ángulos seleccionados (grados)
   |-- valor objetivo final f*
   |-- ImrtInstance::reportPlan(s) -> estadísticas clínicas ICRU-83
```

9.2 Reportes Clínicos ICRU-83 (reportPlan)

Tras la optimización, main.cpp invoca reportPlan() sobre la instancia para generar un informe clínico completo conforme al estándar ICRU-83:

Métrica	Fórmula / Descripción
Dmin, Dmean, Dmax	Dosis mínima, media y máxima por órgano (Gy)
D95, D5, D2	Dosis cubriendo el 95%, 5% y 2% del volumen del órgano
V95 (cobertura)	Fracción del PTV con dosis $\geq$ 95% de la prescripción
Conformity Index (CI)	V_{Rx} / V_{PTV} (volumen con dosis $\geq$ prescripción / volumen PTV)
Homogeneity Index (HI)	$(D2 - D98) / D_{Rx}$ (dispersión de dosis en PTV)
DVH CSV	Curvas DVH por órgano exportadas a dvh.csv
Tabla de restricciones	DVH goals vs. valores obtenidos, marcando OK o VIOLACIÓN

10 Resumen de Diseño y Patrones Arquitectónicos

10.1 Patrones de Diseño Utilizados

Patrón	Uso en EMILI-IMRT
Template Method	LocalSearch define el esquema; subclasses sobrescriben variantes de búsqueda
Strategy	Neighborhood, Perturbation y Acceptance son estrategias intercambiables
Builder	ImrtBuilder construye componentes IMRT desde tokens CLI
Iterator	NeighborhoodIterator permite recorrer vecindades con for-range
Polymorphism	Problem, Solution, Termination permiten extensión específica de dominio
Sparse Matrix	DoseEntry evita almacenar la matriz Dij completa (ahorro de memoria)
Grammar-based Config	Gramáticas XML + irace/GGA para síntesis automática de algoritmos

10.2 Relación entre Componentes Principales

Árbol de dependencias simplificado

```
main.cpp
|-- GeneralParserE
    |-- EmBaseBuilder -> construye LocalSearch, ILS, Tabu, VNS, VND
    |-- ImrtBuilder   -> construye Problem, InitSol, Neighborhood, Perturbation

BaopProblem (ImrtFmoSolver)
|-- ImrtInstance (datos clínicos, matrices Dij dispersas)
|-- OSQP         (resolución QP para cada subconjunto de ángulos)

Metaheurística (ILS ejemplo)
|-- LocalSearch  (First/BestImprovement sobre AngleSwapNeighborhood)
|-- Perturbation (RandomAnglesPerturbation)
|-- Acceptance   (ImrtImproveAccept)
```

10.3 Comparativa de Modos de Operación

Aspecto	Modo FMO Puro	Modo BAO + FMO
Variable de decisión	Intensidades $x[j]$ in R^n	Subconjunto de K ángulos
Espacio de búsqueda	Continuo (R^n)	Combinatorio ($C(n,K)$)
Función objetivo	Penalización cuadrática	QP exacto vía OSQP
Vecindad	Shift/Swap de beamlets	Intercambio de ángulos
Coste por evaluación	$O(\text{entradas Dij})$	$O(\text{QP solve}) \gg \text{FMO aprox.}$
Solución inicial	Zero / Uniform / Random	FirstK / RandomK ángulos
Perturbación	k beamlets aleatorios	p ángulos aleatorios
Compilación	Siempre disponible	Requiere WITH_OSQP=ON

Extensibilidad del Sistema

Para agregar un nuevo metaheurístico: heredar de LocalSearch en emilibase.h y registrar el token en EmBaseBuilder.

Para agregar una nueva vecindad IMRT: heredar de ImrtNeighborhood (imrt.h) e implementar computeStep() / reverseLastMove(), luego registrar en ImrtBuilder.

Para soportar un nuevo formato de instancia: extender ImrtInstance::loadFromDirectory() con la lógica de detección y carga correspondiente.